TECNOLÓGICO NACIONAL DE MÉXICO

INSTITUTO TECNOLÓGICO DE TIJUANA

SUBDIRECCIÓN ACADÉMICA

DEPARTAMENTO DE SISTEMAS Y COMPUTACIÓN

SEMESTRE:

ENERO - AGOSTO 2026

CARRERA:

INGENIERÍA EN SISTEMAS COMPUTACIONALES

MATERIA:

PATRONES DE DISEÑO

UNIDAD A EVALUAR:

III

NOMBRE Y NÚMERO DE CONTROL DEL ALUMNO:

VELAZQUEZ MORALES LUCERO - 22210362

NOMBRE DE MAESTRO(A):

GUERRERO LUIS MARIBEL

Patrón Composite

El patrón Composite es un patrón de diseño estructural que permite componer objetos en estructuras de árbol para representar jerarquías de parte-todo.

El objetivo es permitir que los clientes traten de manera uniforme a los objetos individuales y a los complejos. Es especialmente útil para modelar y trabajar con estructuras complejas.

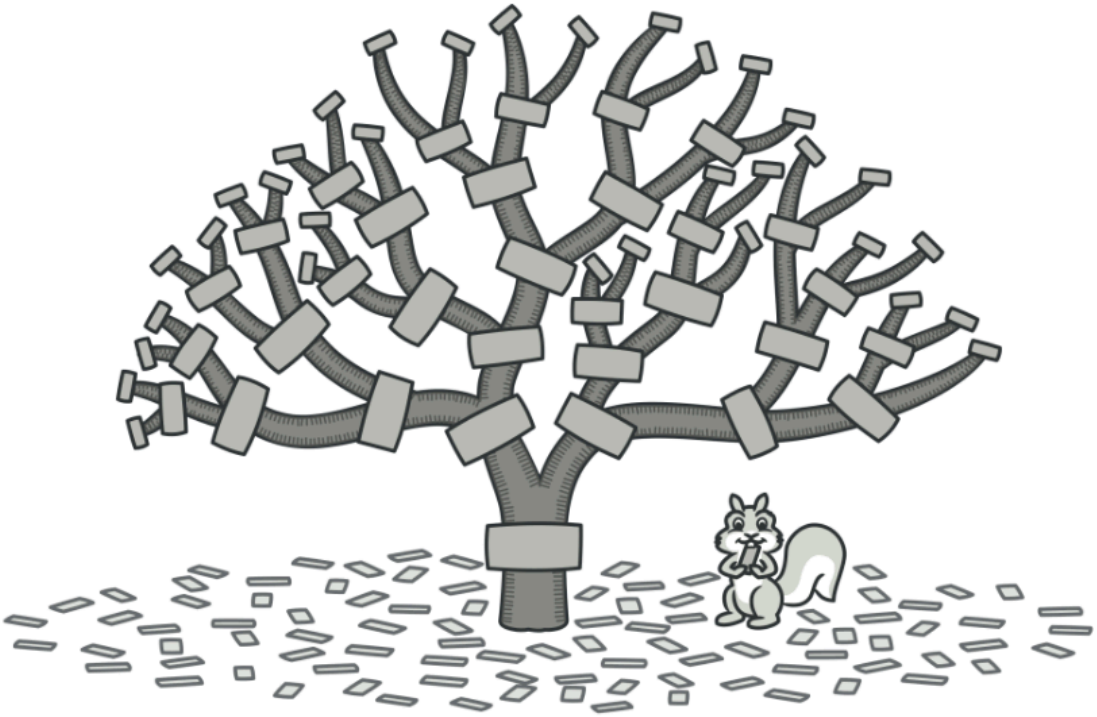
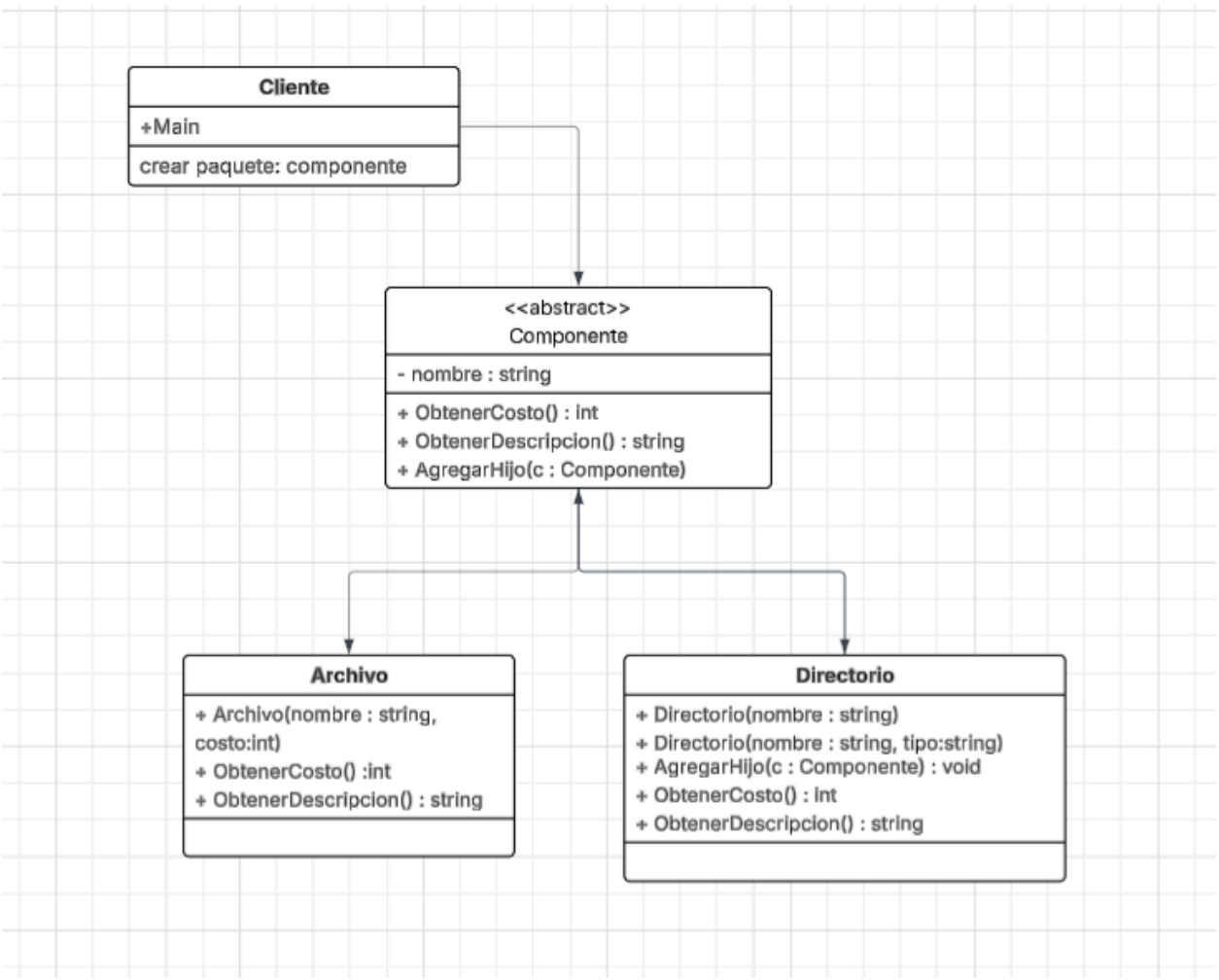
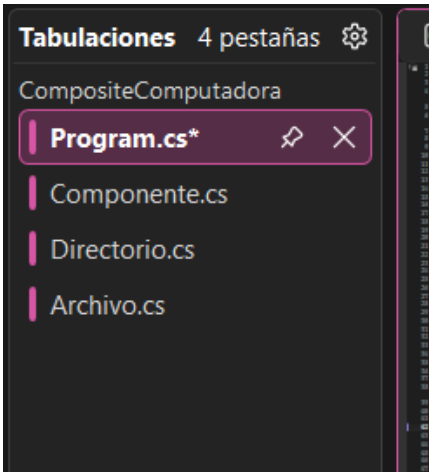


Diagrama UML

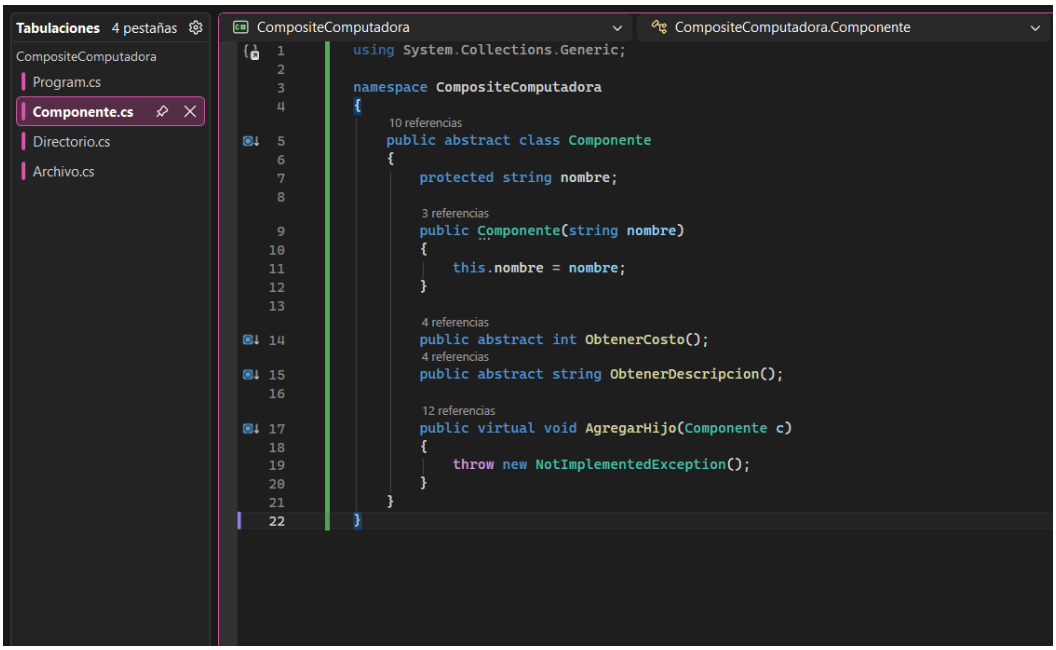


Código



Componente

La clase componente es la base del patrón composite define métodos comunes como obtener el costo y la descripción



Directorio

Directorio es un objeto compuesto que puede contener otros componentes y que guarda hijos en una lista que es agregarHijo() que agrega componentes, ObtenerCosto() que suma los costos y obtenerDescripcion que muestra la información

```
using CompositeComputadora;
using System.Collections.Generic;
using static System.Runtime.InteropServices.JavaScript.JSType;

6 referencias
public class Directorio : Componente
{
    private List<Componente> hijos = new List<Componente>();

    1 referencia
    public Directorio(string nombre) : base(nombre) { }

    1 referencia
    public Directorio(string nombre, string tipo) : base(nombre + " (" + tipo + ")") { }

    12 referencias
    public override void AgregarHijo(Componente c)
    {
        hijos.Add(c);
    }

    3 referencias
    public override int ObtenerCosto()
    {
        int total = 0;
        foreach (var hijo in hijos)
        {
            total += hijo.ObtenerCosto();
        }
        return total;
    }

    3 referencias
    public override string ObtenerDescripcion()
    {
        string resultado = nombre + "\n";
        foreach (var hijo in hijos)
        {
            resultado += "  " + hijo.ObtenerDescripcion() + "\n";
        }
        return resultado;
    }
}
```

Archivo

La clase Archivo representa los elementos individuales del sistema, como CPU, RAM

-public class Archivo : Componente (hereda de componente)

-private int costo; (guarda el precio del componente)

-public Archivo(string nombre, int costo) : base(nombre)(constructor recibe el nombre,costo y base"nombre" que manda el nombre a la clase componente)

-this.costo = costo; (guarda el costo en la variable)

```
using System.Collections.Generic;
using static System.Runtime.InteropServices.JavaScript.JSType;

namespace CompositeComputadora
{
    11 referencias
    public class Archivo : Componente
    {
        private int costo;

        10 referencias
        public Archivo(string nombre, int costo) : base(nombre)
        {
            this.costo = costo;
        }

        2 referencias
        public override int ObtenerCosto()
        {
            return costo;
        }

        2 referencias
        public override string ObtenerDescripcion()
        {
            return nombre + " - $" + costo;
        }
    }
}
```

Aquí se muestra el menu donde el usuario selecciona el tipo de computadora que desea armar

```
Tabulaciones 4 pestañas
CompositeComputadora
Program.cs*
Componente.cs
Directorio.cs
Archivo.cs
CompositeComputadora | CompositeComputadora.Program | Main(string[] args)

4   {
        0 referencias
5       class Program
6       {
            0 referencias
7           static void Main(string[] args)
8           {
9               int opcion;
10
11              do
12              {
13                  Console.Clear();
14                  Console.WriteLine("===== MENU =====");
15                  Console.WriteLine("1. Computadora de gama alto");
16                  Console.WriteLine("2. Computadora de gama Medio");
17                  Console.WriteLine("3. Computadora de gama Bajo");
18                  Console.WriteLine("4. Salir");
19                  Console.Write("Seleccione: ");
20
21                  int.TryParse(Console.ReadLine(), out opcion);
22
23                  switch (opcion)
24                  {
25                      case 1:
26                          ArmarPC("Computadora Cliente de gama alto", "Alta");
27                          break;
28                      case 2:
29                          ArmarPC("Computadora Cliente de gama media", "Media");
30                          break;
31                      case 3:
32                          ArmarPC("Computadora Cliente bajo", "Económica");
33                          break;
34                  }
35
36              } while (opcion != 4);

103 %   0   5   Línea: 23, Carácter: 32   SPC   CRLF   UTF-8 with BOM
```

Aquí se crea la computadora como un objeto compuesto y un contenedor llamado componentes, donde se almacenarán todos los elementos de la computadora

```
39
        3 referencias
40      static void ArmarPC(string nombre, string tipo)
41      {
42          Directorio pc = new Directorio(nombre, tipo);
43          Directorio componentes= new Directorio("Componentes");
44
```

dependiendo del tipo de computadora se asignan diferentes precios a los componentes

```
Tabulaciones 4 pestañas
CompositeComputadora
Program.cs*
Componente.cs
Directorio.cs
Archivo.cs
CompositeComputadora | CompositeComputadora.Program | ArmarPC(string nombre, string tipo)

43          Directorio componentes= new Directorio("Componentes");
44
45
46          int precioCPU1, precioCPU2;
47          int precioRAM1, precioRAM2;
48          int precioDISCO1, precioDISCO2;
49          int precioTEC1, precioTEC2;
50          int precioMOU1, precioMOU2;
51
52
53          if (tipo == "Alta")
54              {
55                  precioCPU1 = 8000; precioCPU2 = 6000;
56                  precioRAM1 = 5000; precioRAM2 = 3000;
57                  precioDISCO1 = 4000; precioDISCO2 = 2000;
58                  precioTEC1 = 1500; precioTEC2 = 800;
59                  precioMOU1 = 1200; precioMOU2 = 600;
60              }
61          else if (tipo == "Media")
62              {
63                  precioCPU1 = 3000; precioCPU2 = 2800;
64                  precioRAM1 = 1500; precioRAM2 = 800;
65                  precioDISCO1 = 2000; precioDISCO2 = 700;
66                  precioTEC1 = 800; precioTEC2 = 500;
67                  precioMOU1 = 600; precioMOU2 = 300;
68              }
69          else // ECONOMICO
70              {
71                  precioCPU1 = 2000; precioCPU2 = 1500;
72                  precioRAM1 = 800; precioRAM2 = 500;
73                  precioDISCO1 = 700; precioDISCO2 = 400;
74                  precioTEC1 = 400; precioTEC2 = 250;
75                  precioMOU1 = 300; precioMOU2 = 150;
76              }
77

103 %   0   5   Línea: 137, Carácter: 13   SPC   CRLF   UTF-8 with BOM
```

aquí se agregan los componentes como objetos tipo archivo dentro del componentes usando el método AgregarHijo

```
Tabulaciones 4 pestañas
CompositeComputadora
Program.cs*
Componente.cs
Directorio.cs
Archivo.cs
CompositeComputadora | CompositeComputadora.Program | ArmarPC(string nombre, string tipo)

            precioTEC1 = 400; precioTEC2 = 250;
            precioMOU1 = 300; precioMOU2 = 150;
        }

        Console.Clear();
        Console.WriteLine($"Armando {nombre}\n");


        Console.WriteLine("CPU:");
        Console.WriteLine($"1. Intel i5 (${precioCPU1})");
        Console.WriteLine($"2. Ryzen 5 (${precioCPU2})");
        int cpu = int.Parse(Console.ReadLine());

        if (cpu == 1)
            componentes.AgregarHijo(new Archivo("Intel i5", precioCPU1));
        else
            componentes.AgregarHijo(new Archivo("Ryzen 5", precioCPU2));


        Console.WriteLine("\nRAM:");
        Console.WriteLine($"1. 16GB (${precioRAM1})");
        Console.WriteLine($"2. 8GB (${precioRAM2})");
        int ram = int.Parse(Console.ReadLine());

        if (ram == 1)
            componentes.AgregarHijo(new Archivo("RAM 16GB", precioRAM1));
        else
            componentes.AgregarHijo(new Archivo("RAM 8GB'
                                                class System.String
                                                Represents text as a sequence of UTF-16 code units.


        Console.WriteLine("\nAlmacenamiento:");
        Console.WriteLine($"1. SSD 1TB (${precioDISCO1})");
        Console.WriteLine($"2. HDD 500GB (${precioDISCO2})");
        int disco = int.Parse(Console.ReadLine());

103 %   0  5                                   Línea: 137, Carácter: 13   SPC   CRLF   UTF-8 with BOM
```

aquí se agrega el componentes a la computadora, formando una estructura jerárquica tipo árbol

```
        componentes.AgregarHijo(new Archivo("Mouse Genius", precioMOU2));

    pc.AgregarHijo(componentes);


    Console.Clear();
    Console.WriteLine("===== RESULTADO =====");
```

resultado

```
C:\Users\lucer\source\repos\(

===== MENU =====
1. Computadora de gama alto
2. Computadora de gama Medio
3. Computadora de gama Bajo
4. Salir
Seleccione:
```

```
C:\Users\lucer\source\repos\(

Armando Computadora Cliente de gama alto

CPU:
1. Intel i5 ($8000)
2. Ryzen 5 ($6000)
1

RAM:
1. 16GB ($5000)
2. 8GB ($3000)
1

Almacenamiento:
1. SSD 1TB ($4000)
2. HDD 500GB ($2000)
1

Teclado:
1. Logitech ($1500)
2. Redragon ($800)
1

Mouse:
1. Logitech ($1200)
2. Genius ($600)
1
```

```
C:\Users\lucer\source\repos\(

===== RESULTADO =====
Tipo: Alta
Nombre: Computadora Cliente de gama alto

Descripción:
Computadora Cliente de gama alto (Alta)
  Componentes
  Intel i5 - $8000
  RAM 16GB - $5000
  SSD 1TB - $4000
  Teclado Logitech - $1500
  Mouse Logitech - $1200


Costo total: $19700
```

Conclusión

En esta práctica se utilizó el patrón composite para representar una computadora como una estructura donde hay componentes individuales y grupos de componentes

esto permitió manejar todos los elementos de la misma manera y calcular el costo total fácilmente. Además entendi cómo organizar objetos en forma de árbol para hacer el programa más claro y estructurado.

Referencias

Composite

https://refactoring.guru/es/design-patterns/composite

Patron composite|composición de objetos en estructuras de árbol

https://codeandosimple.com/design-patterns-composite

¿Cuáles son las mejores marcas de teclado?

https://intercompras.com/blog/mejores-marcas-de-teclado/